Série d'exercices : langage Dart

Types, conditions, boucles, fonctions et programmation orientée objet

Pr. Abdelfattah Abassi
Faculté des Sciences et Techniques de Settat – Université Hassan I
Année universitaire 2026–2027

Consignes générales

— Réaliser les exercices dans DartPad (`https://dartpad.dev`) ou dans un environnement Dart.

— Pour chaque programme, tester plusieurs valeurs, y compris les cas limites.

— Sauf indication contraire, écrire un programme complet contenant une fonction `main()`.

— Le travail demandé consiste à écrire le code et à vérifier son résultat. Aucun corrigé n'est fourni dans cette série.

Exercice 1 – Variables, types et expressions

(6 points)

1. Déclarer des variables représentant le nom d'un étudiant, son âge, sa moyenne et son statut d'admission. Choisir un type Dart adapté à chaque donnée.
2. Déclarer une variable `prixHT` de type `double` et une variable `tva` de type `double`. Calculer le prix TTC avec une TVA de 20 %.
3. Déclarer une liste de cinq notes de type `double`. Afficher la première note, la dernière note et le nombre de notes.
4. Expliquer la différence entre `var`, `final` et `const` à l'aide d'un petit exemple pour chacun.
5. Écrire un exemple utilisant une variable nullable de type `String?` et l'opérateur `??` pour fournir une valeur par défaut.

Exercice 2 – Conditions et énumérations

(8 points)

1. Écrire un programme qui lit ou définit un nombre entier et affiche s'il est positif, négatif ou nul.
2. Écrire une fonction `String mention(double moyenne)` qui renvoie :
   — `Très bien` si la moyenne est supérieure ou égale à 16 ;
   — `Bien` si elle est supérieure ou égale à 14 ;
   — `Assez bien` si elle est supérieure ou égale à 12 ;
   — `Passable` si elle est supérieure ou égale à 10 ;
   — `Insuffisant` dans les autres cas.
3. Écrire une fonction `bool estBissextile(int annee)` qui indique si une année est bissextile. Une année est bissextile si elle est divisible par 400, ou si elle est divisible par 4 mais pas par 100.
4. Définir `enum Niveau { licence, master, doctorat }`. À l'aide de `switch`, associer à chaque niveau une description (`Bac+3`, `Bac+5` ou `Bac+8`).

Exercice 3 – Boucles et calculs

(8 points)

1. Afficher les entiers de 1 à 20 avec une boucle `for`.
2. Calculer la somme des entiers de 1 à n, où n est un entier positif.
3. Afficher la table de multiplication d'un entier n, de $1 \times n$ à $10 \times n$.
4. Écrire une fonction `bool estPremier(int n)` qui renvoie `true` si n est premier et `false` sinon. Tester notamment les valeurs 0, 1, 2, 9 et 17.
5. À l'aide d'une boucle, calculer la somme des éléments de la liste `[8, 12, 15, 6, 17]` puis sa moyenne.
6. Réécrire le parcours de la liste précédente avec une boucle `for-in`. Indiquer pourquoi les indices ne sont pas nécessaires dans cette version.

Exercice 4 – Fonctions

(10 points)

1. Écrire `double celsiusVersFahrenheit(double c)` en utilisant la formule

$$F = C \times 1{,}8 + 32.$$

2. Écrire `int carre(int x)` sous forme classique avec `return`, puis sous forme fléchée avec `=>`.
3. Écrire `void saluer(String nom)` qui affiche un message de bienvenue. Appeler cette fonction pour trois noms différents.
4. Écrire `String creerMessage({required String nom, int age = 18})`. La fonction doit renvoyer un message contenant le nom et l'âge. Tester un appel avec l'âge par défaut et un appel avec un âge fourni.
5. Écrire `double prixTTC({required double prixHT, double tva = 0.20})`. Calculer le prix TTC avec la TVA par défaut, puis avec une TVA de 10 %.
6. Écrire une fonction récursive `int factorielle(int n)`. Traiter le cas de base et tester les valeurs de 0 à 6.
7. Expliquer la différence entre un paramètre et un argument à partir d'un appel à `carre(5)`.

Exercice 5 – Traitement des listes

(8 points)

On considère la liste suivante :

```
final nombres = [3, 8, 12, 5, 20];
```

1. Utiliser `where` pour obtenir les nombres supérieurs ou égaux à 10.
2. Utiliser `map` pour obtenir une liste contenant le carré de chaque nombre.
3. Combiner `where` et `map` pour obtenir les carrés des nombres strictement supérieurs à 5.

4. Utiliser `reduce` pour calculer la somme de tous les nombres.
5. Utiliser `any` pour vérifier si la liste contient un nombre supérieur à 18, puis `every` pour vérifier si tous les nombres sont positifs.
6. Expliquer le rôle de `toList()` dans les expressions utilisant `where` ou `map`.

Exercice 6 – Première classe et objets

(8 points)

On souhaite représenter des livres dans une petite bibliothèque.

1. Créer une classe `Livre` possédant les attributs `titre`, `auteur` et `nbPages`.
2. Ajouter un constructeur qui initialise les trois attributs.
3. Ajouter une méthode `void afficher()` qui affiche les informations du livre.
4. Ajouter un getter `estLong` qui renvoie `true` si le livre contient plus de 300 pages.
5. Dans `main()`, créer deux livres différents, afficher leurs informations et indiquer si chacun est long.
6. Modifier la classe pour utiliser des paramètres nommés dans le constructeur, avec `titre` et `auteur` obligatoires.

Exercice 7 – Encapsulation et validation

(8 points)

Créer une classe `CompteBancaire` qui respecte les règles suivantes :

1. L'attribut `titulaire` est une chaîne de caractères.
2. Le solde est stocké dans un attribut privé `_solde`, initialisé à zéro.
3. Un getter `solde` permet de consulter le solde sans le modifier directement.
4. La méthode `deposer(double montant)` ajoute le montant seulement s'il est strictement positif.
5. La méthode `retirer(double montant)` retire le montant si celui-ci est positif et inférieur ou égal au solde. Elle renvoie `true` si le retrait est réalisé, sinon `false`.
6. Tester un dépôt de 500, un retrait de 200 et une tentative de retrait de 1 000. Afficher le solde après chaque opération.
7. Ajouter une méthode `virer(CompteBancaire autre, double montant)` pour transférer de l'argent vers un autre compte, sans autoriser un solde négatif.

Exercice 8 – Héritage et polymorphisme

(8 points)

1. Créer une classe `Animal` avec un attribut `nom`, un constructeur et une méthode `crier()`.
2. Créer deux classes `Chien` et `Chat` qui héritent de `Animal`. Chaque classe redéfinit la méthode `crier()` pour afficher un son adapté.

3. Créer une liste de type `List<Animal>` contenant au moins un chien et un chat.
4. Parcourir la liste et appeler `crier()` pour chaque animal. Expliquer pourquoi le résultat dépend de l'objet réel.
5. Dans la classe `Animal`, ajouter une méthode `sePresenter()`. La redéfinir dans `Chien` ou `Chat` en appelant d'abord la version du parent avec `super`.

Exercice 9 – Classe abstraite et formes géométriques

(6 points)

1. Définir une classe abstraite `Forme` contenant une méthode abstraite `double aire()`.
2. Créer une classe `Rectangle` avec la longueur et la largeur. Redéfinir `aire()`.
3. Créer une classe `Cercle` avec le rayon. Redéfinir `aire()` en utilisant $A = \pi r^2$.
4. Placer un rectangle et un cercle dans une liste `List<Forme>`, puis afficher l'aire de chaque forme avec une boucle.
5. Expliquer en quelques lignes pourquoi cet exercice illustre le polymorphisme.

Exercice 10 – Mini-projet : gestionnaire de notes

(10 points)

Développer un programme Dart qui gère les notes d'une classe. Le programme doit respecter le cahier des charges suivant :

1. Créer une classe `Etudiant` avec un nom et une liste de notes.
2. Écrire `ajouterNote(double note)`. La méthode accepte uniquement les notes comprises entre 0 et 20 ; sinon, elle affiche un message et ne modifie pas la liste.
3. Ajouter un getter `moyenne`. Si la liste des notes est vide, la moyenne vaut 0.
4. Ajouter un getter `mention` utilisant les seuils de l'exercice 2.
5. Redéfinir `toString()` pour afficher le nom, la moyenne arrondie à deux décimales et la mention.
6. Dans `main()`, créer au moins trois étudiants et leur attribuer plusieurs notes, dont une note invalide afin de tester la validation.
7. Trier les étudiants par moyenne décroissante et afficher le classement.
8. Afficher la moyenne de la classe et le nom de l'étudiant ayant la meilleure moyenne.

Questions de synthèse

Répondre brièvement aux questions suivantes.

1. Quelle est la différence entre une fonction qui renvoie une valeur et une fonction de type `void` ?
2. Quelle est la différence entre une classe et un objet ?
3. À quoi servent un constructeur et le mot-clé `this` ?

4. Pourquoi utilise-t-on un attribut privé et un getter ?
5. Quelle est la différence entre `extends`, `super` et `@override` ?
6. Quelle est la différence entre `where`, `map` et `reduce` ?

Conseil : testez vos programmes avec des valeurs normales, des valeurs limites et des valeurs invalides.